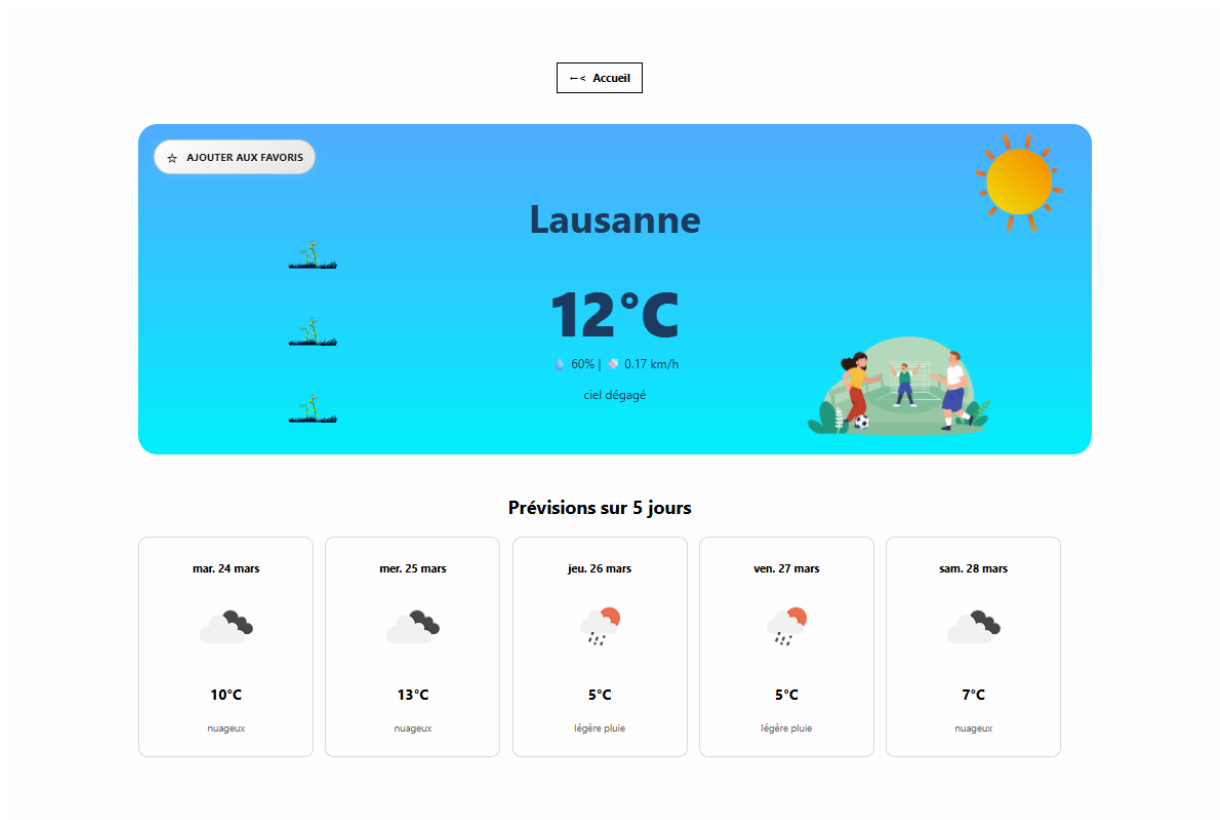


Fun Weather App



Projet Pré-Tpi - 2026

Ahmet Karabulut SI-CA2a

Table des matières

Analyse préliminaire	4
1.1 Introduction	4
1.2 Glossaire	4
1.3 Objectifs	5
1.3.1 Objectifs techniques	5
1.3.2 Objectifs fonctionnels	5
1.3.3 Objectifs pédagogiques	5
1.4 Planification initiale	6
1.4.1 Détail des Sprints	6
1.4.2 Visualisation du calendrier	7
2 Analyse / Conception	8
2.1 Technologies utilisées	8
2.2 Outils d'aide	9
2.3 Maquettes	9
2.4 Stratégie de test	13
2.4.1 Choix de l'outillage : Pourquoi Vitest ?	13
2.4.2 Nomenclature : Pourquoi .spec.js au lieu de .test.js ?	13
2.4.3 Configuration du projet	14
2.4.4 Structure et Co-location (__tests__)	15
2.5 Risques techniques	16
2.5.1 Identification	16
2.5.2 Analyse (Matrice des risques)	16
2.5.3 Évaluation et Priorisation	16
2.5.4 Traitement (Stratégies de réduction)	16
2.5.5 Suivi et contrôle	17
2.6 Planification	17
2.7 Dossier de conception	18
2.7.1 Architecture modulaire des composants visuels	18
2.7.2 Routage	19
2.7.3 Gestion des appels API avec Axios	19
2.7.4 Logique de rendu dynamique des scènes	19

2.7.5	Gestion de l'Interface Utilisateur (UI/UX)	20
2.7.6	Sécurité des données	21
3	Réalisation.....	21
3.1	Dossier de réalisation	21
3.1.1	Structure du repository	22
3.2	Construction de la documentation	24
3.3	Résultat des tests effectués	25
3.3.1	Tests unitaires	25
3.3.2	Tests manuels :	25
3.4	Couverture des tests.....	26
3.4.1	Méthodologie de test des composants	27
3.4.2	Limites des tests automatisés et tests manuels	27
3.4.3	Perspectives d'amélioration	28
3.5	Erreurs restantes	28
3.6	Liste des documents fournis	28
4	Conclusions	28
4.1	Objectifs atteints / non-atteints	28
4.2	Points positifs / négatifs	29
4.3	Difficultés particulières.....	30
4.4	Suites possibles pour le projet.....	30
5	Annexes.....	32
5.1	Résumé du rapport du TPI	32
5.2	Sources – Bibliographie.....	33
5.3	Journal de travail	34
5.4	Manuel d'installation	35
5.5	Manuel d'utilisation	35
5.6	Archives du projet.....	36

Analyse préliminaire

1.1 Introduction

Ce projet, nommé « Fun Weather App », est réalisé dans le cadre du pré-TPI. Il consiste à développer une application météo interactive qui combine des données techniques et une interface utilisateur ludique.

Description du projet L'objectif est de créer une interface web permettant de consulter la météo d'une ville tout en affichant des animations contextualisées selon le temps qu'il fait (par exemple, des enfants jouant sous la neige ou des feuilles s'envolant avec le vent). Le projet repose sur l'intégration de l'API OpenWeatherMap et l'utilisation du framework Vue3 pour assurer une gestion fluide et moderne de l'interface.

1.2 Glossaire

API (Interface de Programmation d'Application) : Ensemble de protocoles et de définitions permettant à deux logiciels de communiquer entre eux. Dans ce projet, elle est utilisée pour récupérer les données météorologiques en temps réel via OpenWeatherMap.

Asynchrone (Programmation) : Méthode permettant d'exécuter des opérations (comme un appel API) en arrière-plan sans bloquer l'exécution du reste du code ou l'interface utilisateur.

Composants (Vue.js) : Éléments d'interface réutilisables et autonomes qui structurent l'application Vue 3.

DOM (Document Object Model) : Interface de programmation qui représente la structure d'un document HTML. Vue 3 manipule le DOM de manière réactive pour mettre à jour l'affichage.

Déploiement : Processus consistant à transférer l'application finale sur un serveur distant pour la rendre accessible en ligne aux utilisateurs.

ES6+ (ECMAScript 6) : Version moderne de JavaScript apportant des fonctionnalités avancées comme les classes, les promesses et les fonctions fléchées.

Framework : Structure logicielle offrant un ensemble d'outils et de bibliothèques pour faciliter le développement d'applications. Vue 3 est le framework principal utilisé ici.

GSAP (GreenSock Animation Platform) : Bibliothèque JavaScript performante dédiée à la création d'animations complexes et fluides sur le web.

LottieFiles : Format de fichier d'animation basé sur JSON qui permet de diffuser des animations vectorielles légères et de haute qualité.

OpenWeatherMap : Service en ligne fournissant des données et des prévisions météorologiques mondiales via une API.

Réactivité : Capacité d'un système à mettre à jour automatiquement l'interface utilisateur dès que les données sources (état) sont modifiées.

UX (Expérience Utilisateur) : Qualité de l'interaction entre l'utilisateur et l'application, incluant la fluidité, l'esthétique et la facilité d'utilisation.

1.3 Objectifs

Ce chapitre énumère les objectifs principaux du projet « Fun Weather App ». Ces points serviront de base pour évaluer la réussite du projet lors de la phase finale.

1.3.1 Objectifs techniques

- Intégration d'API : Récupérer avec succès des données météorologiques en temps réel et des prévisions multi-jours via l'API OpenWeatherMap.
- Utilisation de Vue 3 : Mettre en place une architecture réactive utilisant le framework Vue 3 pour la gestion de l'état global de l'application et la manipulation du DOM.
- Animations dynamiques : Implémenter des scènes illustratives (pluie, neige, vent, soleil) qui se déclenchent automatiquement en fonction des conditions météo reçues.
- Gestion des erreurs : Assurer la robustesse de l'application en gérant les erreurs liées aux appels API (ex: ville non trouvée, problème de connexion).

1.3.2 Objectifs fonctionnels

- Recherche de localisation : Permettre à l'utilisateur de saisir manuellement le nom d'une ville pour obtenir des informations instantanées.
- Affichage des prévisions : Présenter de manière claire et structurée les prévisions météorologiques pour les jours à venir (3 à 7 jours).
- Interactivité et UX : Créer une interface utilisateur fluide, visuelle et attractive, facilitant la navigation et la compréhension des données.

1.3.3 Objectifs pédagogiques

- Approfondir les connaissances en JavaScript moderne (ES6+) et en programmation asynchrone.
- Maîtriser le cycle de vie d'un projet web, de la conception des wireframes au déploiement final.

- Apprendre à utiliser des bibliothèques d'animations spécialisées telles que GSAP ou LottieFiles

1.4 Planification initiale

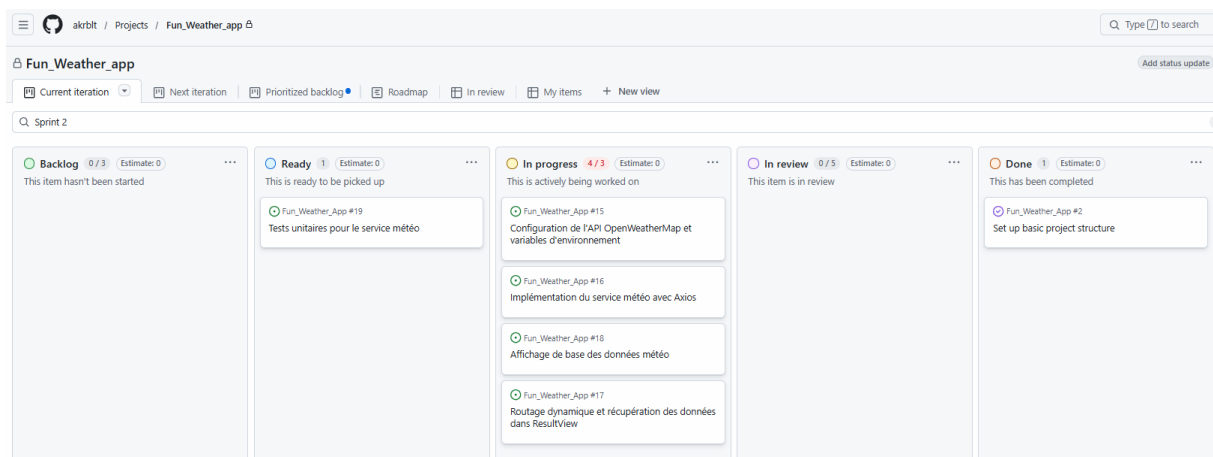
« Le projet est structuré selon une approche agile, découpée en six sprints distincts pour garantir une progression constante. La gestion et le suivi des priorités sont assurés via GitHub Projects, permettant une synchronisation transparente entre le développement et les jalons de la "Fun Weather App". »

1.4.1 Détail des Sprints

Le projet s'étend du début du mois de février jusqu'au début du mois d'avril 2026, selon la répartition suivante :

- Sprint 1 - Configuration du projet & Maquette (3 fév - 9 fév) : Mise en place de l'environnement de développement avec Vue 3 et Git, et conception des wireframes sur Figma ou Balsamiq.
- Sprint 2 - Intégration de l'API Météo (10 fév - 13 fév) : Développement de la logique de récupération des données en temps réel via l'API OpenWeatherMap.
- Sprint 3 - Animations & Expérience Visuelle (23 fév - 2 mar) : Création et intégration des scènes animées (pluie, neige, soleil) à l'aide de bibliothèques comme GSAP ou LottieFiles.
- Sprint 4 - Gestion des erreurs & UX (3 mar - 9 mar) : Optimisation de l'interface utilisateur et mise en place de la gestion des erreurs API pour assurer la stabilité de l'application.
- Sprint 5 – Déploiement et Documentation (13 mar – 1 avr) : Phase intensive de tests, corrections de bugs et mise en ligne de l'application finale.

Voici à quoi ressemble mes kanbans pour chaque Sprint:



1.4.2 Visualisation du calendrier

Cette planification, bien qu'établie dès le départ, sera revue et affinée à l'issue de la phase d'analyse pour s'adapter aux éventuelles contraintes techniques rencontrées lors de l'implémentation.

No	Sprint	Titre	Estimé
1	1	Imaginer fonctionnalité	2h
2	1	Imaginer maquette	2h
3	1	Réunion avec le chef du projet	1h
4	1	Github repo & project & user story	2h
5	1	Installation logicielle	1h
6	1	Maquette	2h
7	1	Documentation sprint 1	2h
8	2	Creation de composants search bar.vue	2h
9	2	Codé en main page	1h
10	2	Ajouter fonctionnalité search bar, bouton , ville suggéré dans main page	2h
11	2	Implemation unit test avec premier test reussi	2h
12	2	Configuration .env pour apikey	0.5h
13	2	Implémentation du Service Météo	2h
14	2	Intégration Axios & Appels API (Current Weather)	3h
15	2	Gestion du State (Pinia)	4h
16	2	Formatage des données (Date, Température Celsius)	2h
17	2	Documentation & Revue Sprint 2	2h
18	3	Recherche & Sélection assets (Lottie/SVG/Iconnes)	3h
19	3	Cherche LottiFile	3h
20	3	Création Scène : Soleil /Ciel Dégagé	5h
21	3	Création Scène : Pluie / Orage	5h

22	3	Création Scène : Neige / Brouillard	5h
23	3	Transition dynamique entre les météo	4h
24	4	Design responsive (Mobile/ Desktop)	4h
25	4	Gestion des erreurs (City not found , API down)	3h
26	4	Indicateurs de chargement(Skeletons/Spinners)	2h
27	5	Vérification de Tests unitaires	4h
28	5	Optimisation des performances	2h
29	5	Déploiement (Vercel/ Netlify)	1.5h
30	6	Rédaction du rapport technique final	4h
31	6	Préparation de la démo / Présentation	5h
		Total	84h

2 Analyse / Conception

2.1 Technologies utilisées

Pour répondre aux exigences techniques et assurer une interactivité ludique, les technologies suivantes ont été sélectionnées :

- Vue 3 : Framework principal utilisé pour la création d'une interface réactive et la gestion efficace des composants.
- JavaScript (ES6+) : Langage de base pour la logique asynchrone et les appels API.
- HTML5 / CSS3 : Pour la structure sémantique et la mise en forme stylisée de l'application.
- OpenWeatherMap : API publique utilisée pour la récupération des données météorologiques mondiales.
- GSAP / LottieFiles : Bibliothèques spécialisées pour l'intégration des animations fluides (pluie, neige, vent).

2.2 Outils d'aide

Le développement est soutenu par une suite d'outils modernes, chacun jouant un rôle spécifique dans la réussite et l'organisation du projet :

VS Code (Visual Studio Code) : C'est l'éditeur de texte (IDE) principal utilisé pour l'écriture du code. Il est choisi pour sa légèreté et ses extensions qui facilitent le développement avec Vue 3, permettant de détecter les erreurs en temps réel.

Git / GitHub : Ces outils servent à la gestion de version. Git permet de sauvegarder l'historique des modifications, tandis que GitHub héberge le code en ligne. Cela sécurise le travail contre toute perte de données accidentelle.

GitHub Projects : Utilisé pour la gestion de projet. Cet outil permet de créer un tableau de bord (style Kanban) pour suivre l'avancement des tâches (À faire, En cours, Terminé). Il aide à respecter les délais de chaque Sprint.

Vite : C'est l'outil de construction (build tool) qui accompagne Vue 3. Il permet de lancer le projet très rapidement durant le développement et d'optimiser les fichiers finaux pour qu'ils soient les plus légers possible pour l'utilisateur.

Postman : Cet outil est essentiel pour le test des API. Avant d'écrire le code pour récupérer la météo, Postman permet de vérifier que l'API d'OpenWeatherMap répond correctement et de comprendre exactement la structure des données reçues (température, vent, humidité).

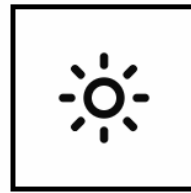
Figma / Balsamiq : Outils dédiés à la conception des maquettes. Balsamiq permet de dessiner rapidement des plans simples (wireframes) pour structurer l'interface, tandis que Figma aide à définir le design visuel précis et l'expérience utilisateur (UX).

Vercel / Netlify : Plateformes choisies pour le déploiement. Elles permettent de mettre l'application en ligne automatiquement dès que le code est mis à jour sur GitHub, rendant le projet accessible via un lien URL.

2.3 Maquettes

Conformément à la planification du Sprint 1, j'ai réalisé une phase de prototypage à l'aide de l'outil Figma. Cette étape de conception graphique est essentielle pour définir l'ergonomie de l'application avant d'entamer le développement technique sous Vue 3. L'objectif est de créer une interface 'Fun' et intuitive qui respecte la hiérarchie des informations météorologiques tout en laissant place aux animations dynamiques. Ces maquettes Figma servent de plan directeur pour l'intégration des composants et la gestion du responsive design.

Écran d'accueil



WEATHER APP

Recherchez la météo de votre ville

**CHERCHER**

VILLES SUGGÉRÉES:



Paris



London



Oslo



Chicago



Tokyo



Istanbul



New York



Berlin

Weather App

Recherche à une Ville

Recherchez la météo de votre ville

**CHERCHER**

Weather App

NEIGE

Enfant 1

Enfant 2

Bonhomme
de neige

← RETOUR À LA RECHERCHE

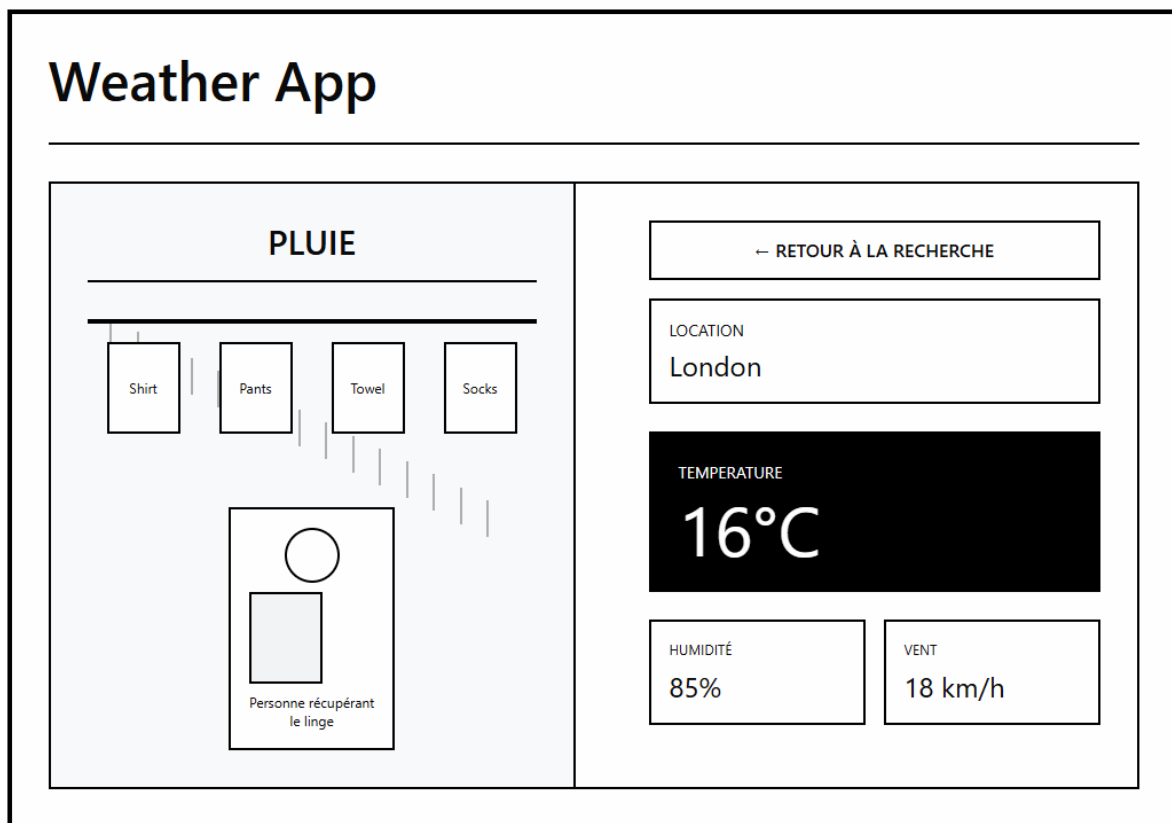
LOCATION
Oslo

TEMPERATURE
-2°C

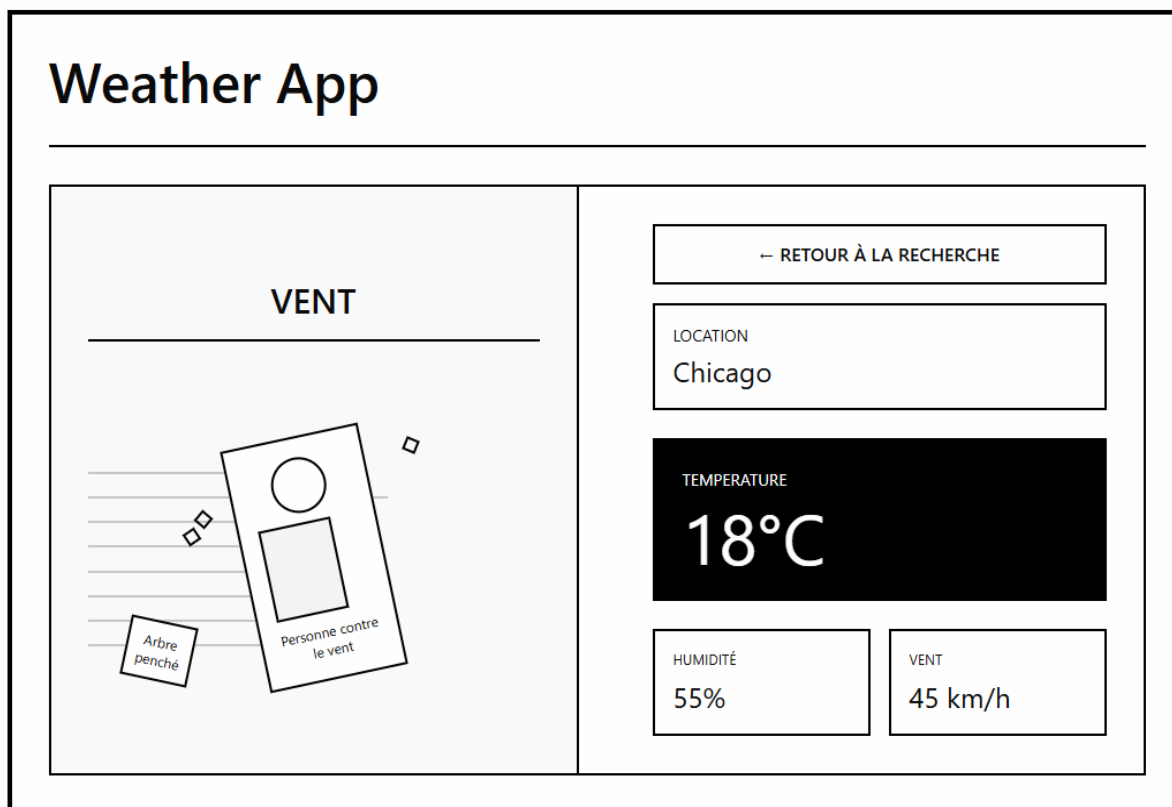
HUMIDITÉ
75%

VENT
8 km/h

Écran des Résultats (Pluie)



Écran des Résultats (Vent)



2.4 Stratégie de test

Cette section décrit la méthodologie utilisée pour garantir la fiabilité de la « Fun Weather App ». Je teste manuellement les fonctionnalités dans mon navigateur (Chrome ou Firefox) et j'utilise des outils de tests automatisés adaptés à l'écosystème Vue 3. La stratégie adoptée se rapproche du BDD (*Behaviour Driven Development*), consistant à définir le comportement attendu (par exemple : « l'affichage d'une icône de pluie lorsque l'API retourne un temps pluvieux ») avant de finaliser le développement. La suite de tests est lancée régulièrement pour s'assurer que l'intégration de nouvelles animations ou de prévisions n'altère pas les fonctions existantes.

2.4.1 Choix de l'outillage : Pourquoi Vitest ?

Pour ce projet basé sur Vue 3, j'ai choisi Vitest comme moteur de test principal au lieu de Jest ou Mocha. Ce choix est motivé par plusieurs facteurs techniques :

- Vitesse d'exécution : Vitest utilise l'infrastructure de Vite, ce qui permet des tests extrêmement rapides grâce au rechargement à chaud (Hot Module Replacement).
- Compatibilité Native : Contrairement à Jest, Vitest partage la même configuration que le projet principal (vite.config.js), évitant ainsi les duplications de réglages.
- Écosystème Vue 3 : Il offre une intégration parfaite avec Vue Test Utils, l'outil officiel pour manipuler les composants Vue dans un environnement de test.

2.4.2 Nomenclature : Pourquoi .spec.js au lieu de .test.js ?

J'ai adopté l'extension .spec.js pour mes fichiers de test. Ce choix s'inscrit dans une approche de Spécification Technique :

- Un fichier .test.js se contente de vérifier si le code fonctionne.
- Un fichier .spec.js sert de document de référence décrivant le comportement attendu (les spécifications) du composant.
- *Exemple* : SearchBar.spec.js définit que "le composant doit émettre un événement lors du clic". Cela facilite la compréhension du rôle de chaque composant pour tout nouveau développeur.

2.4.3 Configuration du projet

- A. Modification de vite.config.js : Pour que les tests puissent simuler un navigateur sans en ouvrir un réellement, j'ai configuré un environnement jsdom. Voici la configuration ajoutée :

```
// helpers/vite.config.js
export default defineConfig({
  plugins: [vue()],
  test: {
    // c'est neccessaire que jsdom pour vitest
    environment: 'jsdom',
    globals: true, // fonction describe, it etc etre global
  }
})
```

- B. Automatisation via package.json : Pour simplifier le flux de travail, j'ai ajouté un script personnalisé. Cela permet de lancer toute la suite de tests avec une simple commande :

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "test": "vitest",
  "preview": "vite preview"
```

Ahmet Karabulut, 6 days ago • Set

- C. Commande : npm run test

```
C:\Users\pr08glt\Desktop\Pré_TPI\Pre_Tpi\fun_weather_app (develop -> origin)
λ npm run test

> fun_weather_app@0.0.0 test
> vitest
```

- D.** Mode Watch : Vitest reste en écoute et relance les tests dès qu'une modification est détectée dans le code source, garantissant un cycle de développement fluide.

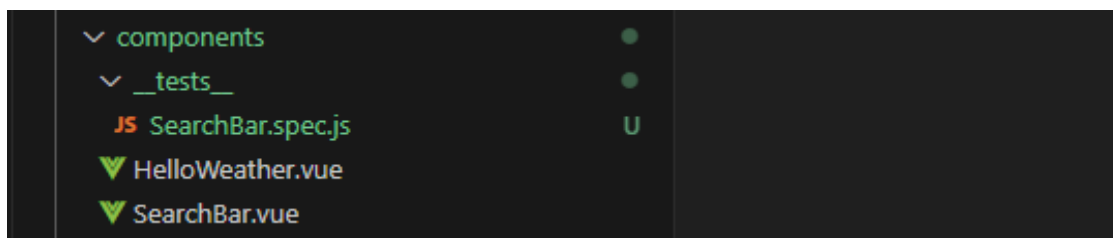
```
RERUN src/components/__tests__/SearchBar.spec.js x1

✓ src/components/__tests__/SearchBar.spec.js (1 test) 27ms
  ✓ SearchBar Test (1)
    ✓ should have a button with the text CHERCHER 25ms

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    11:36:10
Duration    169ms

PASS Waiting for file changes...
press h to show help, press q to quit
█
```

2.4.4 Structure et Co-location (__tests__)



Les tests ne sont pas isolés dans un dossier racine distant, mais placés dans un répertoire `__tests__` à l'intérieur de chaque dossier de composant.

- **Visibilité** : Le préfixe `__` (Dunder => "Double Under score") permet de distinguer visuellement les fichiers d'outillage des fichiers de production dans l'arborescence.
- **Maintenabilité** : En gardant le test à côté de son composant, je m'assure que toute modification du code est immédiatement suivie d'une mise à jour de son test de spécification.

2.5 Risques techniques

La gestion des risques pour ce projet suit un cycle structuré afin d'améliorer la stabilité et les chances de succès, tout en augmentant la confiance dans les solutions techniques choisies.

2.5.1 Identification

L'objectif est de découvrir les risques potentiels avant qu'ils ne surviennent. Les principaux facteurs de risque identifiés sont :

Technique : Difficultés liées à l'apprentissage de la *Composition API* de Vue 3.

Externe : Modification des conditions d'utilisation ou indisponibilité de l'API OpenWeatherMap.

Performance : Ralentissements dus au poids des animations (Lottie/GSAP) sur les appareils mobiles.

2.5.2 Analyse (Matrice des risques)

J'évalue ici la probabilité d'occurrence et l'impact potentiel pour chaque risque afin de définir son niveau de criticité.

Risque	Probabilité	Impact	Niveau de Risque
Indisponibilité de l'API	Faible	Critique	Moyen
Complexité de Vue 3	Moyenne	Élevé	Élevé
Lenteur des animations	Moyenne	Moyen	Moyen
Retard de planification	Faible	Élevé	Moyen

2.5.3 Évaluation et Priorisation

Les risques sont classés par ordre d'urgence pour orienter les efforts de développement

Priorité 1 (Critique) : Maîtrise de Vue 3. Conditionne l'ensemble de l'architecture et le respect des délais dès le Sprint 1.

Priorité 2 (Moyen) : Connectivité API. Indispensable pour le fonctionnement de base (affichage des données).

Priorité 3 (Modéré) : Fluidité des animations. Crucial pour l'esthétique et l'expérience utilisateur (UX).

2.5.4 Traitement (Stratégies de réduction)

Pour chaque risque, une stratégie spécifique est mise en œuvre :

Atténuer : Utilisation du Sprint 1 pour une auto-formation intensive sur Vue 3 afin de sécuriser le développement.

Éviter : Mise en place de données de secours (*fallbacks*) et mise en cache locale pour parer aux coupures réseau ou aux erreurs d'API.

Optimiser : Choix de formats d'animation légers (JSON via Lottie) et tests de performance réguliers pour garantir la fluidité sur mobile.

2.5.5 Suivi et contrôle

La surveillance est continue tout au long du projet :

- Contrôle hebdomadaire : À chaque fin de Sprint (tous les lundis), un point de situation est effectué pour vérifier l'efficacité des mesures prises.
- Ajustement : Si un risque évolue (ex: lenteur constatée), la stratégie est immédiatement adaptée pour le sprint suivant.

2.6 Planification

La liste des tâches est la même qu'au départ, les estimations n'ont pas été modifiées. Afin de comparer ce qui avait été prévu et ce qui s'est réellement passé finalement, j'ai rajouté quelques colonnes. **S-d** signifie Sprint de départ et **S-f** signifie Sprint final (est différent pour les tâches achevée en retard ou en avance). **Le Delta** est le résultat de Temps estimé - Temps passé.

No	S-d	S-f	Titre	Estimé	Passé	Delta	Fin
1	1	1	Imaginer fonctionnalité	2h	2h	0h	03.02.2026
2	1	1	Imaginer maquette	2h	2h	0h	03.02.2026
3	1	1	Réunion avec le chef du projet	1h	2h	-1h	03.02.2026
4	1	1	Github repo & project & user story	2h	3h	-1h	04.03.2026
5	1	1	Installation logicielle	1h	1h	0h	04.03.2026
6	1	1	Maquette	2h	3h	-1h	04.03.2026
7	1	1	Documentation sprint 1	2h	1h	1h	04.03.2026
8	2	2	Création de composants search bar, vue	2h	2h	0h	09.02.2026
9	2	2	Codé en main page	1h	1h	0h	09.02.2026
10	2	2	Ajouter fonctionnalité search bar, bouton, ville suggérée dans main page	2h	3h	-1h	10.02.2026
11	2	2	Implémentation unit test avec premier test réussi	2h	1h	1h	10.02.2026
12	2	2	Configuration .env pour apikey	0.5h	0.5h	0h	10.02.2026
13	2	2	Implémentation du Service Météo	2h	2h	0h	10.02.2026
14	2	2	Intégration Axios & Appels API (Current Weather)	3h	4h	-1h	23.02.2026
15	2	3	Gestion d'API	4h	3h	1h	24.02.2026

16	2	3	Formatage des données (Date, Température Celsius)	2h	2h	0h	24.02.2026
17	2	3	Documentation & Revue Sprint 2	2h	2h	0h	25.02.2026
18	3	3	Recherche & Sélection assets (Lottie/SVG/Iconnes)	3h	4h	-1h	02.03.2026
19	3	3	Cherche LottiFile	3h	2h	1h	03.03.2026
20	3	3	Création Scène : Soleil /Ciel Dégagé	5h	4h	1h	04.03.2026
21	3	3	Création Scène : Pluie / Orage	5h	3h	2h	10.03.2026
22	3	3	Création Scène : Neige / Brouillard	5h	3h	2h	11.03.2026
23	3	3	Transition dynamique entre les météo	4h	4h	0hh	11.03.2026
24	4	4	Ville favori max 5	2h	2h	0h	16.03.2026
25	4	4	Animations pour 5 prévisions du jour	1h	3h	-2h	16.03.2026
26	4	4	Design responsive (Mobile/ Desktop)	4h	2h	2h	17.03.2026
27	4	4	Gestion des erreurs (City not found , API down)	3h	3h	0h	17.03.2026
28	4	4	Indicateurs de chargement(Skeletons/Spinners)	2h	1h	1h	17.03.2026
29	5	4	Vérification de Tests unitaires et manuels	4h	4h	0h	17.03.2026
30	5	5	Optimisation des performances	2h	2h	0h	23.03.2026
31	5	5	Déploiement (Vercel/ Netlify)	1.5h	1.5h	0h	31.03.2026
32	5	5	Rédaction du rapport technique final	4h	9h	-5h	25.03.2026
33	5	5	Préparation de la démo / Présentation	5h	7h	-2h	31.03.2026
			Totaux	84h	90h	-6h	

2.7 Dossier de conception

Le projet "Fun Weather App" repose sur une architecture moderne découplée, où la logique de récupération des données est séparée de la logique d'affichage.

2.7.1 Architecture modulaire des composants visuels

L'application est découpée en composants spécifiques pour chaque ambiance météo, situés dans le dossier @/components/scenes/. Cette approche permet de séparer la logique de calcul (données API) de la logique d'animation (GSAP/Lottie).

Les composants suivants ont été créés et sont chargés dynamiquement :

- SunnyScene.vue : Gère l'affichage des conditions de ciel dégagé avec des animations de lumière et de soleil.
- CloudlyScene.vue : Affiche une ambiance de ciel couvert avec des mouvements de nuages lents.
- RainyScene.vue : Intègre des animations de précipitations (pluie et orages) avec un dégradé de fond sombre.
- SnowyScene.vue : Gère l'affichage des particules de neige et des ambiances de brouillard.
- WindyScene.vue : Scène spéciale déclenchée selon la vitesse du vent, animant des éléments (comme des feuilles ou des lignes de vent) de manière plus rapide.

2.7.2 Routage

L'application utilise Vue Router pour gérer la navigation. J'ai défini deux routes principales :

- Accueil (/) : Affiche la barre de recherche et les suggestions de villes.
- Résultat (/meteo/:ville) : Une route dynamique qui utilise le paramètre :ville pour charger les données correspondantes. Cela permet à l'utilisateur de partager un lien direct vers la météo d'une ville spécifique.

2.7.3 Gestion des appels API avec Axios

Pour la communication avec OpenWeatherMap, j'ai centralisé la logique dans un service dédié (weatherService.js).

- Requêtes asynchrones : J'utilise des fonctions async/await pour garantir une expérience fluide.
- Promise.all : Dans la vue de résultat, j'exécute simultanément l'appel pour la météo actuelle et l'appel pour les prévisions sur 5 jours afin de réduire le temps de chargement total.
- Localisation : Les appels API sont paramétrés avec lang: 'fr' pour obtenir des descriptions textuelles directement en français.

2.7.4 Logique de rendu dynamique des scènes

C'est le cœur "Fun" de l'application. Le choix de l'animation ne repose pas seulement sur un texte, mais sur une logique de calcul (computed properties) :

1. Analyse de l'état : Le script analyse les données de weatherData.

2. Priorisation : Si la vitesse du vent dépasse 25 km/h, la scène WindyScene est priorisée, même s'il y a des nuages. Sinon, l'application bascule entre Sunny, Cloudly, Rainy ou Snowy selon les IDs de l'API.

```
// logic to determine the scene
const weatherMain = computed(()=>{
  if(!weatherData.value) return 'sun'
  const main=weatherData.value.weather[0].main.toLowerCase()
  const wind= weatherData.value.wind.speed
  if(wind > 25) return 'windy'
  if(main.includes('cloud')) return 'cloudly'
  if(main.includes('rain') || main.includes('drizzle')) return 'rainy'
  if(main.includes('snow')) return 'snowy'
})
```

3. Composant dynamique : J'utilise la balise spécialisée <component :is="currentScene" /> de Vue 3. Cela permet de monter et démonter les composants d'animation (SunnyScene, RainyScene, etc.) de manière fluide grâce à une enveloppe <transition name="fade">.

```
<transition name="fade" mode="out-in">
  <component :is="currentScene" :key="weatherMain" />
</transition>
```

```
const sceneMap={
  sun:SunnyScene,
  cloudly:CloudlyScene,
  rainy:RainyScene,
  windy:WindyScene,
  snowy:Snowyscene,
}
```

2.7.5 Gestion de l'Interface Utilisateur (UI/UX)

- Feedback utilisateur : Des états loading et error (ref) sont utilisés pour informer l'utilisateur durant les phases de transition ou en cas de saisie d'une ville inexistante.
- Formatage des données : Une fonction utilitaire formatDate utilise toLocaleDateString pour transformer les timestamps bruts de l'API en dates lisibles .

```
// format date
const formatDate = (dateStr) =>{
  const options = {weekday: 'short', day: 'numeric', month: 'short'};
  return new Date(dateStr).toLocaleDateString('fr-FR', options)
}
```

- Design Visuel : Le CSS utilise des dégradés linéaires (linear-gradient) dynamiques appliqués via des classes liées à l'état météo, changeant ainsi l'ambiance de toute la page (ex: bleu clair pour le soleil, gris sombre pour la pluie).

2.7.6 Sécurité des données

La clé d'accès à l'API est strictement isolée. J'utilise import.meta.env.VITE_WEATHER_API_KEY pour récupérer la clé depuis un fichier d'environnement local. Cette pratique empêche le vol de la clé lors de la publication du code sur des dépôts publics comme GitHub.

3 Réalisation

Produit : Fun Weather App

Version : 1.0.0 (Version initiale)

3.1 Dossier de réalisation

Environnement technique (Matériel et Logiciels) :

- Matériel : Dell PC du CPNV
- Système d'exploitation : Windows 11 Éducation
- Outils : Node.js (v22.18.0), NPM (11.6.2), VS Code.

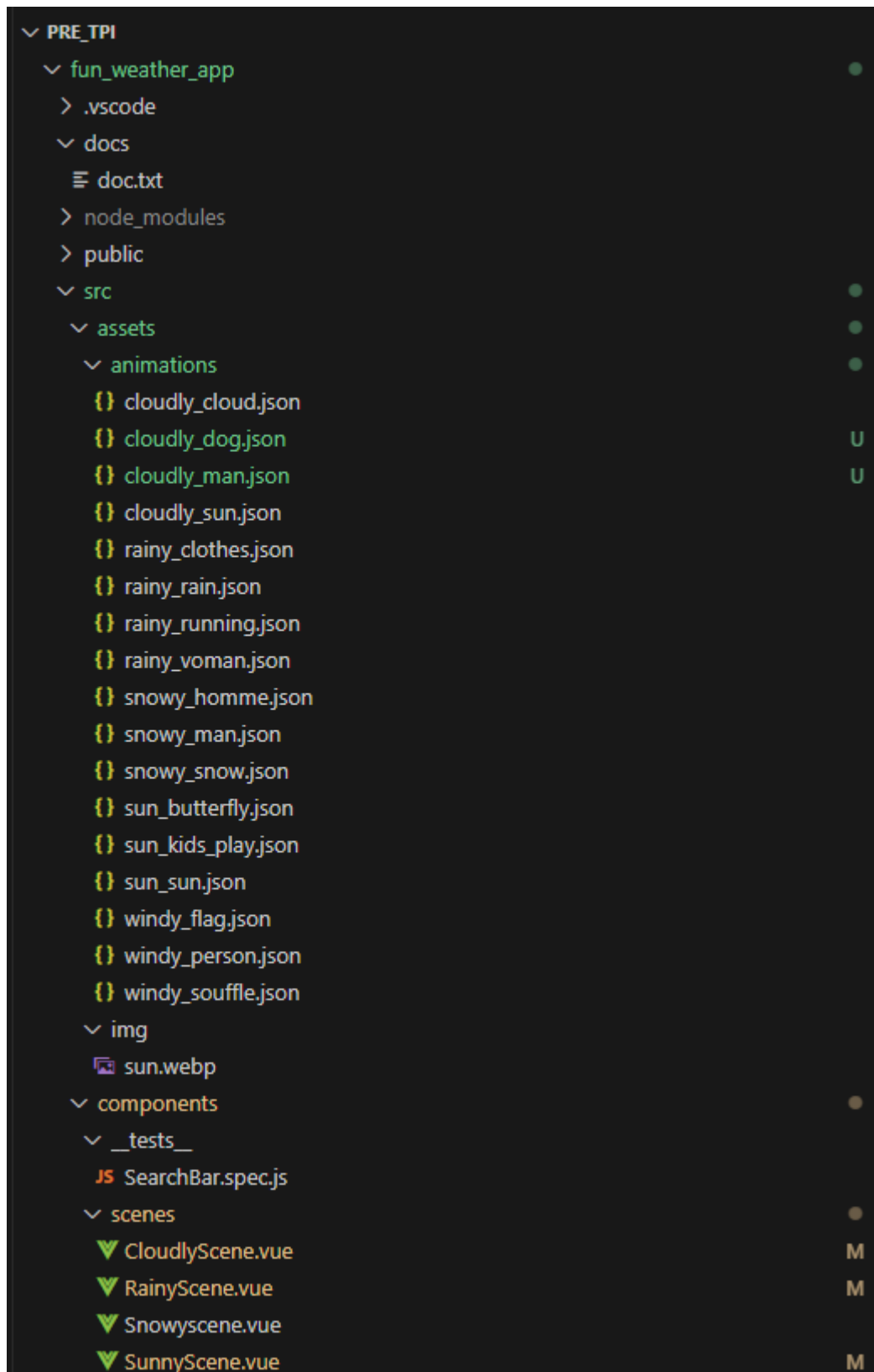
Librairies externes :

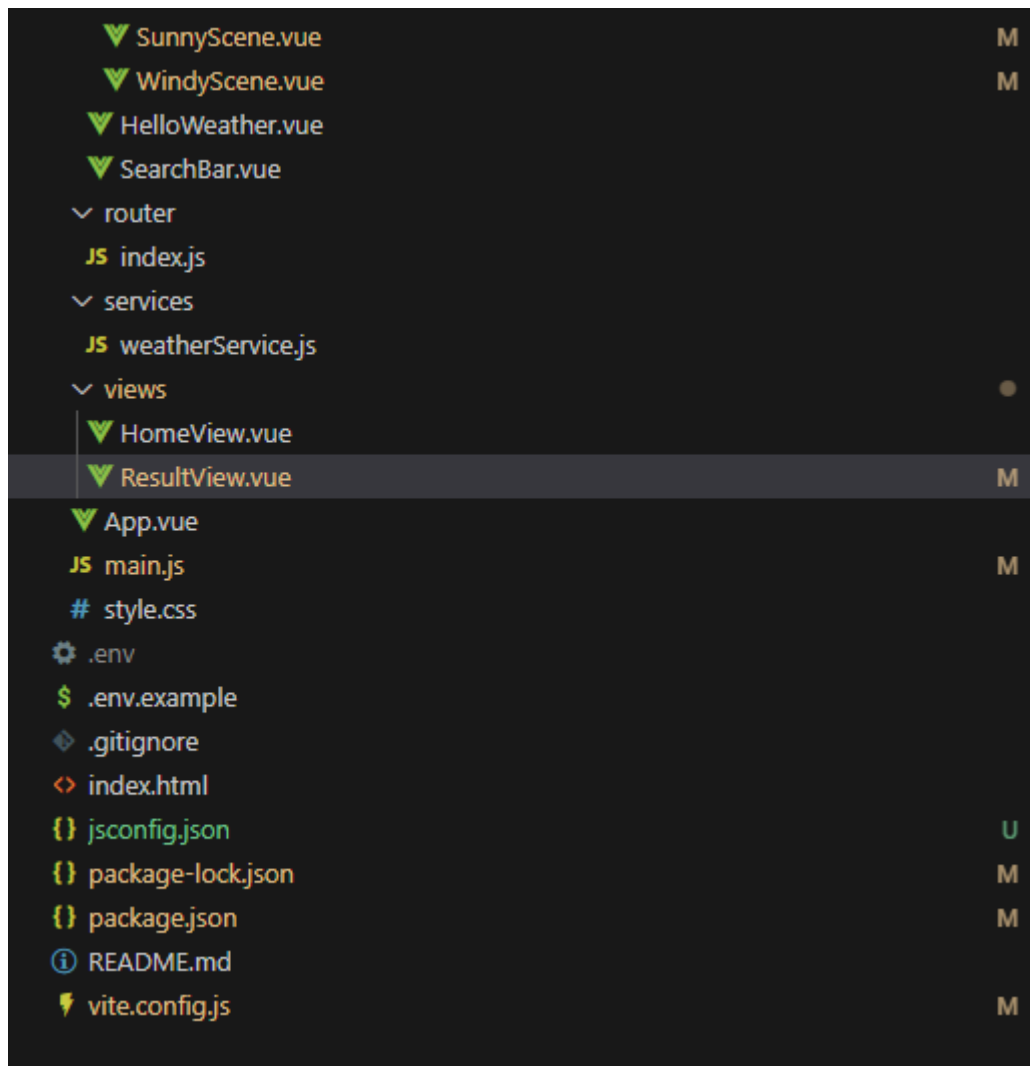
- Vue.js (3.5.24): Framework principal.
- Vite(7.2.4) : Outil de build et serveur de développement.
- Axios(1.13.5) : Gestion des requêtes HTTP vers l'API OpenWeatherMap.
- Vue Router(4.6.4) : Gestion de la navigation entre l'accueil et les résultats.
- Vue3-Lottie(3.3.1): Pour le rendu visuel des scènes météo.
- Vitest(6.0.1) & Vue Test Utils(2.4.6) : Pour les tests unitaires.

3.1.1 Structure du repository

Afin de faciliter la lecture du dossier technique, l'arborescence du projet a été simplifiée. Les répertoires générés automatiquement par l'environnement de développement (tels que `node_modules` ou `dist`) ont été remplacés par des points de suspension (...).

Seuls les fichiers racines et les dossiers spécifiques au développement de la « Fun Weather App » sont présentés ci-dessous. Cette sélection met en lumière les éléments que j'ai personnellement configurés ou implémentés, tels que les services de données, les composants d'animation et la gestion des routes.





3.2 Construction de la documentation

Pour ce projet, j'ai adopté une approche mixte pour la gestion de mes travaux :

- **Code Source (GitHub) :** J'utilise Git uniquement pour le suivi et l'hébergement de mon code source. Cela me permet d'avoir une sauvegarde permanente en ligne et de garantir l'intégrité de mon développement. Bien que je n'utilise pas GitHub pour la rédaction, cela assure une traçabilité technique : chaque étape majeure du code est enregistrée (commit), ce qui permet de justifier l'évolution logique de l'application.
- **Documentation (Word) :** Pour le rapport technique et le journal de travail, j'utilise Microsoft Word. Ce choix me permet de maîtriser la mise en page et la structure visuelle du document. Les versions finales sont exportées en PDF pour garantir la stabilité du rendu lors de la lecture.

Cette séparation me permet d'utiliser l'outil le plus performant pour chaque tâche : Git pour la rigueur technique du code, et Word pour la clarté de la présentation écrite.

3.3 Résultat des tests effectués

3.3.1 Tests unitaires

```
app/node_modules/@vitest/runner/dist/index.js:1653:12)

✓ src/components/__tests__/weatherService.spec.js (2 tests) 9ms

Test Files 7 passed (7)
Tests 13 passed (13)
Start at 14:08:32
Duration 5.88s (transform 828ms, setup 0ms, import 2.34s, tests 282ms, environment 10.31s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

3.3.2 Tests manuels :

Navigation et Recherche

1. Recherche simple : Taper "Paris" dans la SearchBar. Vérifier que l'utilisateur est redirigé vers /resultat/Paris.
2. Retour Accueil : Sur la page de résultat, cliquer sur le bouton "← Accueil". Vérifier le retour à la racine.
3. Suggestions : Cliquer sur une ville suggérée (ex: "Tokyo"). Vérifier que les données de Tokyo s'affichent correctement.
4. Ville inexistante : Taper une ville qui n'existe pas (ex: "Azerty123"). Vérifier que le message d'erreur "Impossible de récupérer les données..." s'affiche.

Système de Favoris (LocalStorage)

5. Ajout favori : Cliquer sur "AJOUTER AUX FAVORIS". Vérifier que l'étoile devient pleine (★) et le texte change.
6. Limite des 5 favoris : Ajouter 5 villes différentes. Essayer d'en ajouter une 6ème. Vérifier que l'alerte "Maximum 5 favoris" apparaît.
7. Suppression favori (Détail) : Cliquer sur un favori déjà actif. Vérifier qu'il redevient "☆ AJOUTER...".
8. Persistance : Ajouter une ville en favori, rafraîchir la page (F5). Vérifier que la ville est toujours marquée comme favori.
9. Affichage Accueil : Ajouter "Londres" en favori, retourner à l'accueil. Vérifier que "Londres" apparaît dans la section "MES FAVORIS".

10. Navigation via Favoris : Cliquer sur un bouton de favori sur l'accueil. Vérifier que les détails de cette ville s'ouvrent.

Affichage Dynamique (Scènes Lottie)

11. Scène Ensoleillée : Rechercher une ville avec un ciel clair. Vérifier que SunnyScene est chargé et que le dégradé est bleu clair.
12. Scène Pluvieuse : Vérifier une ville sous la pluie. S'assurer que les animations de pluie (Lottie) s'affichent par-dessus le fond sombre.
13. Scène Nuageuse : Vérifier que CloudlyScene s'affiche pour "Clouds".
14. Scène Venteuse : Si le vent est > 25 km/h (selon ta logique windy), vérifier que l'animation de vent s'active.
15. Transition de scène : Passer d'une ville ensoleillée à une ville pluvieuse via les suggestions. Vérifier que la transition fade est fluide.

Prévisions (Forecast)

16. Changement de données : Cliquer sur une carte de prévision (ex: météo de demain). Vérifier que l'affichage principal (température, scène, humidité) se met à jour avec les données de ce jour-là.
17. Indicateur de sélection : Vérifier que la carte cliquée dans les prévisions possède bien la classe .active-card (bordure noire).
18. Format de date : Vérifier que les dates des prévisions sont bien formatées en français (ex: "lun. 23 mars").
19. Icônes météo : Vérifier que chaque carte de prévision affiche une icône provenant d'OpenWeatherMap.

Responsive et UI

20. Mode Mobile : Réduire la largeur du navigateur (< 768px). Vérifier que le texte "VOTRE VILLE FAVORITE" disparaît pour ne laisser que l'étoile (comme prévu dans ton CSS @media).

3.4 Couverture des tests

Pour ce projet, j'ai choisi une approche de test adaptée à une application visuelle :

- 80% Tests Manuels : Comme l'application repose beaucoup sur le visuel (animations Lottie, changements de couleurs, design mobile), j'ai testé moi-

même chaque scénario pour m'assurer que l'interface est fluide et agréable à utiliser.

- 20% Tests Unitaires : J'ai utilisé Vitest pour automatiser le test des fonctions logiques les plus importantes, comme la gestion des favoris (LocalStorage) et le tri des données météo.

3.4.1 Méthodologie de test des composants

Pour le développement de mes composants (comme la SearchBar), j'ai suivi une logique rigoureuse :

1. Vérification du rendu : S'assurer que le composant est correctement monté dans le DOM virtuel.
2. Validation des éléments clés : Vérifier la présence des éléments interactifs essentiels (boutons, champs de saisie).
3. Comportement attendu : Tester si le texte affiché correspond exactement aux spécifications (ex: le bouton doit afficher "CHERCHER").

Voici un exemple d'implémentation pour le test unitaire de la barre de recherche :

```
describe('SearchBar Test', () => {  
  it('should have a button with the text CHERCHER', () => {  
    // Create the component virtually  
    const wrapper = mount(SearchBar)  
  
    // Find the button element  
    const buton = wrapper.find('button')  
  
    // Assert that the button text is "CHERCHER"  
    expect(buton.text()).toBe('CHERCHER')  
  })  
})
```

3.4.2 Limites des tests automatisés et tests manuels

Certains aspects de l'application ne sont pas couverts par les tests unitaires actuels et ont nécessité des validations manuelles :

- Interactions avec l'API réelle : Les tests unitaires utilisent souvent des simulations (mocks). La vérification de la connexion réelle avec OpenWeatherMap (gestion des clés API, temps de réponse) a été effectuée manuellement via le navigateur et l'outil Postman.
- Transitions visuelles : La fluidité des animations LottieFile et les transitions <transition> entre les scènes (soleil, pluie, etc.) sont difficiles à tester par le code. J'ai donc validé visuellement que chaque changement de météo déclenchait la bonne ambiance graphique.

- Éléments graphiques SVG : Comme les icônes météo sont des éléments SVG complexes, leur identification précise par les tests automatisés est complexe. Une vérification visuelle a permis de confirmer que l'icône affichée correspond bien à la description textuelle reçue.

3.4.3 Perspectives d'amélioration

Bien que j'ai commencé avec un premier test unitaire réussi pour la `SearchBar`, une amélioration future consisterait à implémenter des tests d'intégration pour vérifier que la ville saisie dans la recherche est correctement transmise au service météo.

3.5 Erreurs restantes

Aucune erreur majeure n'a été détectée. Cependant, une limitation existe : si l'API OpenWeatherMap dépasse son quota de requêtes gratuites, les données peuvent mettre du temps à charger.

3.6 Liste des documents fournis

La livraison finale de la « Fun Weather App » se compose des éléments suivants :

Le rapport de projet (v1.0) : Ce document présent dans sa version finale.

Le code source : L'intégralité du projet est hébergée sur le dépôt GitHub suivant : [Read.me](#)

Le manuel d'installation : Il se trouve dans l'Annexe 5.4 (et également résumé dans le fichier README.md du dépôt). Ce manuel détaille la procédure pour installer le projet à partir de zéro, incluant le clonage du dépôt, l'installation des dépendances via NPM et la configuration de la clé API OpenWeatherMap dans le fichier `.env`.

Le manuel d'utilisation : Il se trouve dans l'Annexe 5.5. Ce guide illustré permet de comprendre comment lancer le serveur de développement local (`npm run dev`) et comment interagir avec l'interface pour rechercher des villes et visualiser les animations météo.

4 Conclusions

4.1 Objectifs atteints / non-atteints

L'évaluation finale de la « Fun Weather App » montre que l'intégralité des objectifs fixés dans le descriptif du projet a été pleinement atteinte. Voici le détail des réalisations par rapport aux exigences initiales :

- Affichage dynamique et ludique : L'objectif principal est rempli. L'application ne se contente pas d'afficher des données chiffrées ; elle les transforme en scènes animées (pluie, soleil, nuages, etc.) via une logique de composants Vue.js.
- Sélecteur de localisation : Le module de recherche est totalement opérationnel. Il permet à l'utilisateur de sélectionner une ville et de déclencher instantanément la mise à jour des données météo.
- Prévisions à moyen terme (3 à 5 jours) : En dessous de l'animation principale, un tableau de bord affiche les prévisions pour les 5 jours suivants, offrant une visibilité claire sur l'évolution climatique.
- Interactivité des prévisions : J'ai implémenté une fonctionnalité permettant de cliquer sur n'importe quel jour de la liste de prévisions pour "prévisualiser" l'animation correspondante dans la scène principale.
- Gestion des favoris (jusqu'à 5 localisations) : Cet objectif a été réalisé en utilisant le localStorage du navigateur. L'utilisateur peut mémoriser ses villes préférées. Ces dernières sont stockées de manière persistante et s'affichent sous forme de raccourcis interactifs sur la page d'accueil.

4.2 Points positifs / négatifs

Points positifs :

- Fiabilité avec Vitest : L'implémentation de tests unitaires avec Vitest et Vue Test Utils a été l'un des points forts de mon développement. Ces tests m'ont permis de garantir la robustesse de mes composants Vue.js et de détecter rapidement les erreurs lors de l'intégration de nouvelles fonctionnalités.
- Qualité documentaire : J'ai investi beaucoup de temps et de soin dans la rédaction de cette documentation sous Word. Mon objectif était de rendre un travail final très qualitatif et soigné. Je pense avoir atteint un niveau de détail et de clarté bien supérieur à mes habitudes.
- Finalisation complète : Un point très positif est d'avoir réussi à livrer 100% des fonctionnalités prévues dans le descriptif, notamment la logique de changement de scènes animées et la gestion de la persistance des favoris.

Points négatifs :

- Gestion du temps en classe : Avec le recul, j'aurais pu mieux optimiser mon temps de travail en classe. Il m'est arrivé de rester bloqué trop longtemps sur l'écriture de certains tests unitaires ou sur des problématiques de rendu visuel que j'aurais pu mettre de côté temporairement pour avancer sur le reste.

- Suivi de l'effort : J'aurais dû effectuer des calculs de totaux d'heures plus réguliers. Cela m'aurait permis de prendre conscience plus tôt de mon avance ou de mon retard réel, au lieu de devoir fournir un effort intensif sur la fin pour tout finaliser.

4.3 Difficultés particulières

La principale difficulté technique de ce projet a été la mise en cohérence des données de l'API avec le rendu visuel des animations.

- Choix technologique : Pour garantir une intégration fluide et performante avec Vue 3, j'ai choisi d'utiliser des fichiers Lottie (LottieFiles). Ces fichiers JSON sont idéaux pour le web car ils sont légers et conservent une qualité vectorielle parfaite.
- Problématique de recherche : Ma vision créative imposait la création de "scènes" spécifiques pour chaque état météo (ensoleillé, pluvieux, neigeux, venteux, etc.) afin de construire un véritable scénario visuel. Cependant, j'ai rencontré de grandes difficultés à trouver des animations Lottie qui soient à la fois de haute qualité et cohérentes entre elles graphiquement.
- Impact sur le planning : J'ai passé un temps considérable à explorer les bibliothèques d'animations pour trouver les fichiers les plus appropriés. Ce processus de sélection a été beaucoup plus long que prévu car il ne suffisait pas de trouver "une" animation, mais de trouver un ensemble de visuels qui s'intègrent harmonieusement dans l'interface de la « Fun Weather App ».

Solution apportée : Malgré le temps passé, cette recherche rigoureuse a permis de garantir une expérience utilisateur immersive où chaque scène météo est immédiatement identifiable et esthétiquement plaisante.

4.4 Suites possibles pour le projet

Le projet « Fun Weather App » a été conçu avec une architecture évolutive qui permet d'envisager plusieurs axes d'amélioration pour le futur :

Enrichissement de l'interface utilisateur (UI) : L'aspect visuel peut encore être approfondi en ajoutant des thèmes dynamiques (mode nuit/jour automatique) ou des transitions plus complexes entre les différentes scènes météo pour une immersion encore plus forte.

Passage à une offre API Premium : L'utilisation de la version payante d'OpenWeatherMap permettrait d'accéder à des données plus précises et variées, telles

que l'indice de qualité de l'air, les alertes météo en temps réel (orages violents, tempêtes) ou des cartes satellites interactives.

Diversification des prévisions temporelles : Actuellement limitée à 5 jours, l'application pourrait être étendue pour proposer plusieurs vues complémentaires :

- Prévisions horaires : Pour une précision heure par heure sur la journée en cours.
- Prévisions étendues : Affichage sur 7 jours (une semaine complète) ou 14 jours.
- Historique météo : Permettre de consulter les données météorologiques des jours passés.

Géolocalisation automatique : Intégrer l'API de géolocalisation du navigateur pour proposer directement la météo de la ville où se trouve l'utilisateur dès l'ouverture de l'application, sans recherche manuelle.

5 Annexes

5.1 Résumé du rapport du TPI

Ce rapport présente le développement de la « Fun Weather App », une application web interactive conçue pour transformer la consultation météo en une expérience ludique et immersive. Réalisée dans le cadre de mon Travail Pratique Individuel (Pré-TPI), cette application combine des données météorologiques réelles avec un rendu visuel dynamique.

Développement technique : L'application est bâtie sur le framework Vue 3 (Vite), choisi pour sa réactivité et sa structure modulaire. Les données sont récupérées en temps réel via l'API OpenWeatherMap. L'innovation visuelle repose sur l'intégration de fichiers Lottie (JSON), permettant d'afficher des scènes animées fluides (pluie, soleil, neige) qui s'adaptent instantanément aux conditions climatiques de la ville recherchée.

Fonctionnalités clés :

- Recherche mondiale : Un sélecteur de localisation permettant d'interroger n'importe quelle ville.
- Prévisions à 5 jours : Une vue détaillée de l'évolution météorologique à moyen terme.
- Système de favoris : Une gestion persistante via localStorage permettant de mémoriser jusqu'à 5 villes préférées.
- Interactivité : La possibilité de visualiser l'animation spécifique de chaque jour de prévision d'un simple clic.

Méthodologie et Qualité : Le projet a suivi une approche rigoureuse incluant des tests unitaires avec Vitest pour garantir la fiabilité des composants. Malgré des défis liés à la recherche de cohérence visuelle pour les animations, l'intégralité du cahier des charges a été respectée.

Conclusion : La « Fun Weather App » démontre la capacité à lier des technologies modernes de développement frontend avec une approche centrée sur l'expérience utilisateur (UX). Le résultat final est une application fonctionnelle, soignée et prête pour de futures évolutions.

5.2 Sources – Bibliographie

Pour la réalisation de ce projet, j'ai utilisé diverses ressources techniques et documentations officielles afin de garantir le respect des standards de développement actuels.

(Frameworks & API)

- Vue.js 3 : <https://vuejs.org/> – Documentation principale pour la structure des composants et la réactivité.
- Vite.js : <https://vitejs.dev/> – Guide de configuration pour l'outil de build et le serveur de développement.
- OpenWeatherMap API : <https://openweathermap.org/api> – Spécifications techniques pour les appels d'API (Current Weather & 5 Day Forecast).
- Vitest : <https://vitest.dev/> – Documentation pour la mise en place des tests unitaires.

Ressources Graphiques & Animations

- LottieFiles : <https://lottiefiles.com/> – Source principale pour la recherche et la sélection des fichiers d'animation JSON.
- Lottie-Vue : <https://github.com/chengqingspring/vue-lottie> (ou la bibliothèque spécifique que tu as utilisée) – Documentation pour l'intégration des animations dans Vue 3.

Outils de Développement

- GitHub : Hébergement du code source et gestion de version (Git).
- Visual Studio Code : Éditeur de texte principal avec les extensions Volar (pour Vue 3) et ESLint.
- Stack Overflow : <https://stackoverflow.com/> – Consulté pour la résolution de problèmes techniques spécifiques liés à JavaScript et CSS.

Intelligence Artificielle & Aide à la Rédaction

- Gemini (Google) : Utilisé comme support à la réflexion algorithmique, à la correction de bogues mineurs et à l'aide à la structuration de la documentation en français.

Aides humaines

M. Hurni: conseils et retours réguliers, réponses à mes questions.

5.3 Journal de travail

Journal de travail Pré-TPI Fun Weather App

S signifie Numéro de Sprint, P (période) est le temps passé sur la tâche en 45 min.

Date	S	P	Description
02.02	1	2	Phase de brainstorming pour définir le périmètre fonctionnel (recherche de localités, gestion des favoris et animations dynamiques).
02.02	1	2	Élaboration des premières maquettes (wireframes) pour structurer la disposition des éléments visuels.
03.02	1	2	Continuer faire de la maquette. J'ai bloqué pendant 30 min. (Balsamique ne marche plus, j'ai reçu une erreur. J'ai fait désinstaller et réinstallé.)
03.02	1	1	Préparation environnement de logiciel. (Installation framework, vue3, projet start)
03.02	1	3	Création du main page avec fonctionnalité search bar
04.02	1	1	Création du main page avec search button (button chercher) et les villes suggères (comme london,tokyo,istanbul,lausanne ... jusqu'à 10 villes). Component SerachBar.vue
04.02	1	1	Documentation sprint 1
09.02	2	1	Implementation Api OpenWeather (service=>weather.service). J'ai créé .env fichier pour garder api clé.
09.02	2	3	Testé d'api et modifier l'information qui viens d'api (anglais vers française et le format date)
10.02	2	1	Creation page Resultview.vue avec meteo du jour . J'ai ajouté seulement icon (soleil,neige, etc) via d'api (pas d'animation)
10.02	2	1	J'ai commencé à ajouter les prévisions météo à 5 jours. J'ai reçu beaucoup de messages d'erreur. En haut de la page, la météo d'un jour différent de celui affiché sous la section des prévisions à 5 jours s'affiche.
10.02	2	3	J'ai commencé à chercher la solution. J'ai essayé qqch mais ça ne marche pas. Et ensuite j'ai trouvé une idée . Il ne marche pas aussi.
10.02	2	1	Documentation sprint 2
11.02	2	2	Cours supprimé
16.02	-	4	Relâches
17.02	-	6	Relâches
18.02	-	2	Relâches
23.02	3	2	J'ai trouvé une idée. Pour la prévision 5 jour, j'ai choisi l'heure pour la météo du 12 :00 h. J'ai appliqué mon projet et il marche .
23.02	3	2	J'ai configuré la connexion entre homepage et resultview. J'ai ajouté bouton retour avec symbole.
24.02	3	1	J'ai configuré prévision 5 jours service avec route : (/forecast)
24.02	3	3	J'ai ajouté les cards pour le prévision 5 jours et j'ai adapté avec le bonne taille
24.02	3	2	J'ai adapté le prévision 5 jours pour l'écran petite taille
25.02	3	2	Documentation sprint 3
02.03	4	2	J'ai cherché et trouvé lotti animations pour dynamique ui. (vue3: Lottie)
03.03	4	5	J'ai cherché lottifile pour le scene soleil et j'ai fait bien adapté et j'ai créé le scene sunnyScene component
03.03	4	1	J'ai testé component SunnyScene et j'ai corrigé l'erreur de s'affiche. (Animation est très grand que l'écran)

04.03	4	1	J'ai cherché lottifile animations pour le scene cloudly. J'ai trouvé 3 animations mais ce n'est pas jolie. J'ai continué à chercher.
04.03	4	1	Documentation sprint4
09.03	4	1	J'ai corrigé le sprint5 pas ce que mes taches n'est pas fini. J'ai du continuer sprint4
09.03	4	2	J'ai trouvé animations pour le scene cloudly et j'ai créé une scene cloudly. Componnet CloudlyScene
09.03	4	1	J'ai testé le component CloudlyScene et j'ai intégré
10.03	4	2	Component RainyScene avec animations lottifile
10.03	4	2	Component WindyScene avec animations lottifile
10.03	4	2	Component SnowyScene avec animations lottifile. Quand j'avais travaillé SunnyScene et CloudlyScene , j'avais appris que comment je peut créer une scene avec lottiFile.
11.03	4	1	J'ai intégré tous les scenes dans le resultview et j'ai testé manuellement.
11.03	4	1	Documentation sprint 4
16.03	4	2	J'ai ajouté localstorage pour 5 villes favori (max 5 ville, s'il essaie à ajouter 6. ville Recevoir une erreur. "Max 5 favoris")
17.03	4	2	J'ai ajouté deux icon pour ville favori. Dans la situation icône change. (Votre ville favorite et Ajouter aux villes favoris '★' : '☆')
17.03	4	2	J'ai ajouté une fonctionnalité pour en click 5 jour sâffiche avec animations
17.03	4	2	Documentation Spring 4 .
18.03	4	2	Je suis absent à cause de mon fils a été opéré par chirurgien à l'hôpital d'Yverdon
23.03	5	2	J'ai terminé le codage. Toute la fonctionnalité c'est bon. J'ai réussi 100% de taches de cahier de charge.
23.03	5	2	J'ai commencé rédiger mon rapport.
24.03	5	6	J'ai continué rédiger mon rapport.
25.03	5	2	J'ai commencé à faire du présentation
30.03	5	4	J'ai relié mon rapport et j'ai terminé ma présentation
01.04	5	2	J'ai envoyé mon rapport et j'ai fait ma présentation.

5.4 Manuel d'installation

L'ensemble de la procédure pour installer et lancer le projet localement est disponible dans le fichier README de mon dépôt GitHub.

Vous pouvez y accéder via le lien suivant : [Fun Weather App](#)

5.5 Manuel d'utilisation

Une fois l'application chargée dans votre navigateur :

- Recherche : Saisissez le nom d'une ville (ex: "Lausanne") dans la barre de recherche au centre de l'écran.
- Affichage : Cliquez sur le bouton "Chercher" (ou l'icône de loupe).

- Animations : Observez l'animation Lottie se mettre à jour instantanément. Si la ville est pluvieuse, une scène de pluie s'animera automatiquement.
- Prévisions : Consultez les prévisions pour les 5 prochains jours qui s'affichent sous l'animation principale.
- Interactivité : Cliquez sur l'une des cartes de prévision pour voir l'animation correspondante s'afficher dans le cadre principal.

5.6 Archives du projet

L'archive finale de la « Fun Weather App » est structurée de la manière suivante :

- fun-weather-app-code.zip : Le code source complet du projet (Répertoire Git), incluant les composants Vue 3, les animations Lottie et les fichiers de configuration (hors node_modules).
- fun-weather-app-documentation.pdf : Ce document constitue le dossier complet et unique du projet. Il regroupe l'intégralité des éléments produits durant le TPI, organisés comme suit :
 - Le Résumé du TPI (5.1) : Synthèse d'une page des objectifs et résultats.
 - Le Rapport de projet : Analyse détaillée, conception technique, réalisation et phase de tests. Cette document.
 - Le Manuel d'Installation (5.4) : Procédure technique pour la mise en place de l'environnement, l'installation des dépendances (npm install) et la configuration de la clé API.
 - Le Manuel d'Utilisation (5.5) : Guide pas à pas pour lancer l'application et utiliser ses fonctionnalités.
 - Le Journal de travail (5.3) : Suivi chronologique détaillé des tâches accomplies chaque jour.